

ons for frequently used technologies and platforms, fostering continuous professional development

- On-boarding and off-boarding is efficient
- Clear career development paths

Quick Start Guide

For Development Teams

- Need a new dependency? Create a Deliverables Request
- Packaging question? Ping @gitlab-build in your issue
- Security concern? Follow our vulnerability management process
- General guidance? Review our workflow documentation

Important: Reach out to the Build team early if your change requires:

- New or updated gems with native extensions
- New or updated external software dependencies
- If you need to do install, update, make, mkdir, mv, cp, chown, chmod, or do compilation in any part of GitLab stack

If a change is reported late in the release cycle or not reported at all, your feature/change might not be shipped within the release.

For New Team Members

- Complete your team onboarding issue using the Team-onboarding template
- Review our workflow documentation
- Set up access to build infrastructure as outlined in your onboarding issue

For Customers and Community

- Installation Issues: Community Forum
- Bug Reports: GitLab Issues with group::build label
- Installation Documentation: Install, Update, and Upgrade pages
- Contributing: See our Community Code of Conduct and contribution guidelines

Communication Channels

- Urgent Issues: Slack gdistribution channel
- Feature Requests: Deliverables Request issue
- General Questions: Ping @gitlab-build in relevant issues
- Merge Request Reviews: Use our merge request workflow for Build team projects

For Support Requests

For complex issues that may require expertise from multiple teams or when it's unclear which team should handle a customer request, use GitLab's unified Request for Help (RFH) process. This process ensures we have a single source of truth and enables better cross-functional collaboration, as many requests actually require expertise from multiple areas of the product.

To open an RFH, refer to the procedures in our how to get help handbook page. This process allows us to track time involved and ensure that the right parties are involved at the correct time.

Responsibilities

Internal Customer Support

- Build Tooling & Infrastructure: Develop and maintain tools that enable efficient component building
- Integration Services: Ensure development team components integrate seamlessly into GitLab packages
- Security Partnership: Collaborate with Security teams to identify and mitigate vulnerabilities
- Dependency Management: Maintain up-to-date, secure dependencies across all components
- Documentation & Self-Service: Create guides enabling development teams to work independently

External Customer Delivery

- Multi-Platform Packages: Build and maintain packages for all supported Linux distributions and architectures
- Container Images: Develop and maintain official GitLab container images
- Cloud Marketplace: Manage listings and integrations with major cloud providers
- Installation Experience: Maintain install, update, and upgrade documentation and processes
- Quality Assurance: Ensure all installation methods are thoroughly tested before release
- License Management: Ensure compliance across all bundled dependencies
- Partner Certifications: Submit to partners for validations and certifications

Team members

The following people are members of the GitLab:Build Team:

```
{{< team-by-manager-slug manager="denisra" team="Build" >}}
```

Primary Projects

omnibus-gitlab - This project creates platform-specific, self-contained GitLab packages and images for self-managed consumption in cloud environments and on-premises hosting.

Cloud Native GitLab provides cloud native containers to deploy GitLab. These containers may be deployed and managed with Helm using GitLab Charts or GitLab Operator on Kubernetes, OpenShift, and Kubernetes compatible container platforms.

Omnibus GitLab project's product outputs

```
mermaid
graph TD
    subgraph Code
        OG --> PKG
        PKG --> DOI
    end
```

```
PKG --> AMI
end
```

```
subgraph Deploy
DOI --> DOK
DOK --> DOC
AMI --> AWS
PKG --> GET
GET --> GCP
GET --> AZURE
GET --> |Future|AWS
GET --> |Future|VMW
end
```

```
OG[Omnibus GitLab]
PKG[Linux Package]
DOI[Container Image]
DOK(Docker)
DOC(Docker Compose)
AMI[Amazon Machine Image]
GET[GitLab Environment Toolkit]
AWS(AWS)
GCP(Google Cloud Platform)
VMW(VMWare)
AZURE(Azure)
```

Cloud Native GitLab project's product outputs

```
mermaid
graph TD
  subgraph Code
    CNG --> HC
    CNG --> GOP
    HC --> GOP
  end
```

```
subgraph Deploy
GOP --> K8s
GOP --> OS
CNG --> DC
HC --> K8s
end
```

```
CNG[Cloud Native GitLab containers]
HC[Helm Chart]
K8s(Kubernetes)
GOP[GitLab Operator]
OS(OpenShift)
DC(Docker Compose)
```

All Projects

Name	Location	Description
Omnibus GitLab	gitlab-org/omnibus-gitlab	Build Omnibus packages with HA support for LTS versions of all supported Linux operating systems such as Ubuntu, Debian, CentOS/RHEL, OpenSUSE, SLES
Docker All in one GitLab image	gitlab-org/omnibus-gitlab/docker	Build Docker images for GitLab CE/EE based on the omnibus-gitlab package
Cloud Native GitLab Containers	gitlab-org/build/CNG	Individual images used by GitLab Helm Charts
AWS images	AWS marketplace	AWS image based on the omnibus-gitlab package
Omnibus GitLab Builder	GitLab Omnibus Builder	Create environment containing build dependencies for the omnibus-gitlab package
Licenses of bundled dependencies	Licenses page on GL Pages	Webpage listing the bundled dependencies in each package along with their license.

Working with the community

The install and upgrade process is one of the first features that systems administrators experience when working with GitLab.

As a result, the projects managed by the Build team have a high level of engagement by the user-base. The GitLab community is made up of more than just code contributors; users logging issues and feature requests are constantly pushing us forward and helping create a better experience.

We strive for the following in our public projects:

1. Uphold our Community Code of Conduct.
1. Enable GitLab's mission that everyone can contribute..
1. Show our work in public.
1. Recognize and thank contributors for their work.
1. Respect contributors donated time by providing a timely review turnaround time.

Working with Open Source communities

The open core of GitLab is built on top of thousands of open source dependencies. These dependencies and their communities are important to the GitLab strategy, and working with these dependencies is an essential part of the projects the Build team maintains.

We strive to:

1. Consider the impact of our work on the open source communities that we benefit from.
1. Promote the importance of these open source communities within GitLab.
1. Raise issue with any decision that goes against our stewardship promises.
1. Find opportunities to contribute back the changes we make.

Public by default

All work carried out by the Build team is public. Some exceptions apply:

- Work has possible security implications - If during the course of work security concerns are no longer valid, it is expected for this work to become public.
- Work is done with a third party - Only when a third party requests that the work is not public.
- Work has financial implications - Unless financial details can be omitted from the work.
- Work has legal implications - Unless legal details can be omitted from the work.

Some of the team work is carried out on our development server at dev.gitlab.org. Infrastructure overview document lists the reasons.

Unless your work is related to the security, all other work is carried out in projects on GitLab.com.

If you need to submit a sensitive issue, please use confidential issues.

If you are unsure whether something needs to remain private, check with the team's Engineering Manager.

Onboarding and offboarding

In addition to general company on-boarding and off-boarding, the Build team has its own process to get new team members up to speed more quickly.

If you are starting with your onboarding, open an issue in Build team issue tracker, select Team-onboarding template and assign the issue to yourself.

Going through the steps noted in the issue should be your top priority, higher than the general company on-boarding issue. This is because items in team on-boarding are specific to your role and it will allow you to get up-to-speed quicker.

Off-boarding should be carried out by the Engineering Manager of the team, using the appropriate issue template in the same issue tracker.

Infrastructure and maintenance

As part of the team responsibilities, the team owns the maintenance of infrastructure used for day to day work.

For list of nodes and description of the maintenance tasks, see the [infrastructure and maintenance page](#).

Team workflows

General engineering workflow applies to the Build team. Since we are working across multiple projects, our team workflow is further explained and summarized on the [Build workflow page](#).

Further reading

The following important areas of the GitLab Handbook impact how we work and are worth reading.

- General engineering workflow page
- How to reinforce our values
- Continue to serve smaller users (internal only)
- Promises to our Open Source community
- How to follow our Product Principles
- Principles of our company strategy
- Effective & Responsible Communication Guidelines
- Test Platform in Distribution group

Work/life harmony

Working all-remote and asynchronous first

offer flexibility in how team members approach their workday. Team members must make choices on how best to balance work time with other areas of life.

For new team members, the following resources provide examples on how to focus their time:

- How team members approach their day
- Blog post: A day in life of a remote worker
- The option of a non-linear workday
- GitLab handbook: Work/life Harmony

The following GitLab Handbook areas are key in maintaining a healthy work/life balance.

- Family and Friends First, work second
- Combating burnout, isolation, and anxiety in the remote workplace
- Recognizing Burnout

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/demo.md

Weekly Demo

Distribution team members present demos related to work they've accomplished (or are developing) and share context. Special guests from outside the team also occasionally present to raise awareness and answer questions on specific subjects.

Demo recordings are publicly available on YouTube to spread knowledge, technologies and solutions the team creates.

Goals

The goals of our demos are to:

1. Spread awareness of current issues impacting our customers and team
1. Solicit feedback from the team to improve our solutions and related documentation
1. Identify gaps in our collective understanding of environments, components, processes and

dependencies

1. Improve visibility of the work our team does

Presenter

The team collaboratively selects the demo presenter during the weekly team meeting from a pool of:

1. Issues labeled Demo-worthy label
1. Suggestions during the weekly team meeting
1. Volunteers from either team or non-team members

The demo presenter is responsible for making sure the demo meeting invite is rescheduled to the proper time.

The presenter is expected to:

1. Prepare any required environmental and related dependencies ahead of the demo
1. Include a demo synopsis in the meeting notes with key component/environment versions, etc. for reproducibility
1. Record the demo locally and upload to YouTube Distribution Team Demos
1. Limit distractions and keep professionalism high by muting slack and presenting in a demo conducive location
1. Follow up on concerns raised during the demo and/or provide links to known issues identified

Attendees

Attendees are expected to:

1. Ask questions - attendee participation is encouraged, but not required
1. Document questions/problems that arise in the meeting notes document
1. Mute yourself while not actively participating to avoid distractions
1. Be internal attendees from Distribution with other teams always welcome

Demo sessions are recorded and uploaded to YouTube GitLab Unfiltered Distribution Demos and are publicly available.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/k8s-release-support-policy.md

The Distribution team's support policy for various Kubernetes and OpenShift releases for the GitLab Helm chart and Operator.

Definitions

Definitions for terms used in this policy.

Kubernetes release

An official minor number release of Kubernetes. Releases can be found at [<https://kubernetes.io/releases/>](https://kubernetes.io/releases/). Kubernetes has official releases 3 times a year roughly 4 months apart which are supported with bug fixes and security patches for 1 year.

OpenShift release

An official minor number release of OpenShift. Releases can be found at [<https://access.redhat.com/support/policy/updates/openshift>](https://access.redhat.com/support/policy/updates/openshift). OpenShift has official releases 3 times a year roughly 4 months apart which are supported with bug fixes and security patches for 1 year.

OpenShift is based on Kubernetes and is functionally very similar. Unless otherwise stated, we treat them as identical for this document.

Supported Kubernetes release

A Kubernetes release that is officially supported by GitLab.

Deprecated Kubernetes release

A Kubernetes release that was previously supported by GitLab and for which we still run minimal testing in our CI/CD environment. For example, with kubeval, kubeconform, or other similar tools. GitLab will likely work on this Kubernetes release, although some features may not.

Qualifying Kubernetes release

A qualifying release is a release that we are testing (with possible porting work) and are adding to our end-to-end E2E testing in our CI/CD system. After a release is qualified, it then becomes an officially supported release. Over time, it will then proceed to a deprecated release.

Unsupported Kubernetes release

An unsupported release is either:

- A new release that has not yet been qualified.
- A previously-supported or deprecated release that is known to be broken.

Constraints

We have several constraints that limit the Kubernetes and OpenShift releases that we can support, either officially or as deprecated releases.

KubectI

Version 1.X.Y of kubectI (where X is the Kubernetes minor version and Y is the Kubernetes patch level) is only supported on Kubernetes cluster versions 1.X-1 to 1.X+1.

API support

Operator relies on backwards compatibility for the Kubernetes API. Kubernetes guarantees backwards compatibility for N-3 versions, so this constraint is less of an issue than for kubectl support.

Kubernetes release schedule

Kubernetes releases 3 times a year with roughly 4 months between releases. Each Kubernetes release is supported for 12 months.

GitLab qualification

The work involved in creating a new cluster to support a new release and adding CI/CD support for it is significant. There is also work in adapting API changes in GitLab Chart and Operator. It takes roughly 3 months to qualify GitLab on a new Kubernetes release.

Cloud provider and Kubernetes flavor release schedules

Various Cloud providers that provide Kubernetes. We will test on only two, GKE and EKS, because of resource constraints and with the understanding that testing on these providers will likely allow us to extrapolate support on other providers running the same Kubernetes releases.

OpenShift is a Kubernetes flavor often run on on-premises infrastructure. There is enough demand and enough idiosyncrasies for OpenShift that we test, qualify, and support it similarly to the cloud providers.

Google Kubernetes Engine (GKE)

GKE is a Kubernetes cloud provider.

GKE supports each release in their Kubernetes release cycle for 14 months. These releases roughly correspond to the Kubernetes release cycle with some lag due to the need for GKE to qualify the release. There are generally 4-5 releases available in the rapid and regular channels with the oldest versions no longer getting patch release or security fixes. The stable channel does not have the newest Kubernetes minor version release so it generally has 4 versions with the oldest no longer receiving maintenance.

See GKE release schedule for details.

Elastic Kubernetes Service (EKS)

EKS is a Kubernetes cloud provider.

EKS follows the Kubernetes release schedule like GKE and Openshift. Each release is supported for 14 months.

See EKS version
for details.

Azure Kubernetes Service (AKS)

AKS is a Kubernetes cloud provider.

AKS follows the Kubernetes release schedule like GKE and Openshift. Each release is supported for 12 months. The latest AKS release usually lags one release behind the latest Kubernetes release.

See AKS Kubernetes release calendar
for details.

OpenShift

OpenShift is a Kubernetes flavor.

Openshift supports each release in their release cycle for 12-18 months. Like GKE, their releases are roughly 4 months apart and roughly correspond to the Kubernetes release cycle. The latest OpenShift release usually lags two releases behind the latest Kubernetes release.

See Openshift versioning for
details.

GKE auto-upgrades

GKE will auto-upgrade clusters that are out of support.

Kubernetes and OpenShift release support policy

The distribution team's support policy for GitLab Helm chart and Operator on Kubernetes and OpenShift releases.

Officially-supported releases

GitLab supports a moving subset of all releases of Kubernetes and OpenShift as new versions are released and old versions are no longer supported by the corresponding providers.

Kubernetes officially supported releases

GitLab will officially support three minor releases of Kubernetes: N, N-1, and N-2. N is either:

- The latest released minor version of Kubernetes, if we have finished qualifying it.
- The next most recent version, if we have not finished or started qualifying the most recent version.

We use the term N+1 for the next upcoming release that may or may not yet be under qualification. GitLab will officially support three minor releases of Kubernetes -- N, N-1, and N-2. N is either the latest released minor version of Kubernetes (if we have finished qualifying it) or the next most recent version (if we have not finished or started qualifying the most recent version). We use the term N+1 for the next upcoming release that may or may not yet be under qualification. For example, if the current releases available are 1.28, 1.27, 1.26, 1.25 and we have not qualified release 1.28, then N would be 1.27 and we would officially support releases 1.25, 1.26, and 1.27 as shown in this table.

Release	Reference	Supported
1.28	N+1	No
1.27	N	Yes
1.26	N-1	Yes
1.25	N-2	Yes

Once 1.28 is qualified, 1.29 becomes N+1 and we add 1.28 to the list of officially supported releases. The latest existing supported release (1.25) becomes deprecated (or possibly unsupported).

Release	Reference	Supported
1.29	N+1	No
1.28	N	Yes
1.27	N-1	Yes
1.26	N-2	Yes
1.25	N/A	Deprecated

Officially-supported OpenShift releases

GitLab will officially support four minor releases of OpenShift -- N, N-1, and N-2 and N-3. Like Kubernetes, N is either:

- The latest released minor version of OpenShift, if we have finished qualifying it.
- The next most recent version, if we have not finished or started qualifying the most recent version.

Again, we use the term N+1 for the next upcoming release that may or may not yet be under qualification. For example, if the current releases available are 4.14, 4.13, 4.12, 4.11 and we have not qualified release 4.15, then N would be 4.14 and we would officially support releases 4.14, 4.13, 4.12, and 4.11 as shown in this table.

Release	Reference	Supported
4.15	N+1	No
4.14	N	Yes

4.13	N-1	Yes	
4.12	N-2	Yes	
4.11	N-2	Yes	

Deprecated releases

We will accept community contributions to fix issues in deprecated releases as long as those fixes do not break supported versions. We may also provide unofficial support for these releases as time and resources allow.

Note that GitLab does not strictly follow the Kubernetes project deprecation schedule.

Kubernetes release support life-cycle

A Kubernetes release goes through the following distribution Kubernetes release support life-cycle:

```

mermaid
graph TD
  A[New release] --> B[In qualification]
  B --> C[Supported]
  C --> D[Deprecated]
  D --> E[Unsupported]
```

New release

A new Kubernetes release that has not yet started qualification for GitLab support.

In qualification

The GitLab distribution team has started qualification of a new release. Qualification includes:

- Conforming to API changes where needed
- Provisioning infrastructure
- Adding CI/CD pipeline
- Adding E2E tests

A complete list of qualification tasks can be found at [Kubernetes release qualification issue template](#)

Supported

Qualification is complete.

Deprecated

A previously supported release can become deprecated for the following reasons:

- The Kubernetes release reaches end-of-life.
- It is the oldest supported release and we have more than three supported releases.

Deprecated releases are supported as a best effort as time and resources allowed by the Distribution team. Releases continue as deprecated as long as they continue to pass kubeval, kubeconform, or other validation tool testing in our CI system and have no known breaking issues. We will consider community contributions as long as these changes do not break support for officially supported and newer deprecated releases.

Unsupported

A release becomes unsupported for the following reasons:

- The release is deprecated and known to be broken (usually because it no longer passes our validation tests).
- The release has been end-of-life for more than one year.

New Kubernetes release support timeline

Newer (N+1) Kubernetes releases will be supported within 3 months of their release.

Kubernetes patch level

We assume that GitLab is backwards and forwards-compatible with patch-level releases of a given Kubernetes minor-level release. That is, if we qualify Kubernetes release 1.N.3, then we can assume qualification for all releases of 1.N.x.

Architectures

Kubernetes can run on several physical architectures. GitLab supports a subset of these architectures due to resource constraints.

x86-64

Our base architecture is x86-64. We support this architecture for all supported Kubernetes releases.

arm64

We do not currently do E2E testing on ARM64. We should add this test on at least

one Kubernetes release before making ARM64 officially supported.

kubectrl version

Where practical, the latest GitLab Helm charts should ship with N-1 version of kubectrl. This will allow us to officially support both N and N-2 Kubernetes release.

CI/CD testing requirements

End-to-End (E2E) testing

The N Kubernetes release gets E2E testing for both x86-64 and ARM64 (when we decide to support it). E2E testing is currently done using our current charts and operator pipelines with the `review<clusterversion>`, `reviewspecs<clusterversion>` (charts only), and `qa<clusterversion>` jobs.

Smoke testing

We do smoke testing for our N, N-1, and N-2 Kubernetes releases. Smoke testing is done in our pipelines with the `reviewvcluster<clusterversion>` jobs. This test deploys the GitLab Helm chart or Operator to a virtual cluster.

Validation testing

Validation testing is done using `kubeconform` or a similar validation tool. GitLab chart pipelines runs `kubconform` in the ``Validate <clusterversion>`` jobs.

GitLab Operator does not currently have a validation test that we can use for this purpose. We will add a corresponding validation test for deprecated releases once we stop doing E2E testing for all releases.

Deprecated release testing

Deprecated releases are tested for regressions using validation testing. As new releases are qualified, a new validation test job is created. Supporting deprecated release then consist of not removing older versions of the validation testing jobs. Once a validation test job breaks due to some incompatibility, we either fix the incompatibility if the deprecation policy allows, or remove that release from the list making it an unsupported Kubernetes release.

Public-facing documentation

We document supported and deprecated Kubernetes releases in tables at:

- GitLab chart
- GitLab Operator

These tables are updated every time we change the list of our supported and/or deprecated releases for both GitLab Helm chart and Operator.

New release process

The process used to add support for a new release and to deprecate older releases is detailed in the issue template.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/merge_requests.md

Introduction

Merge Requests are the responsibility of all Distribution Engineers. For the most part, we follow the engineering process for code review.

Workflow

The Distribution team uses the Reviewers feature in the code review process. The process looks like this:

1. Author opens a merge request in a project.
1. When ready for review:
 - If the merge request included an automatic Reviewer Roulette comment like this one, after the danger-review CI/CD job has finished:
 - The Author assigns the review directly to the reviewer suggested by the Reviewer Roulette and applies the "workflow::ready for review" label using the command "@gitlab-bot ready @username-of-someone".
 - If not:
 - The Author applies the "workflow::ready for review" label.
 - A reviewer checks the Distribution merge requests ready for review dashboard per their priority regularly, and then picks up the merge request when they are able to work on it by adding themselves under the Reviewers section, and adding the "workflow::in review" label.
1. Once approved, the Reviewer adds a Maintainer under the Reviewers section for final review/merge. At this point, the Reviewer may decide whether to keep themselves listed as a Reviewer or to remove themselves (so the MR no longer appears in their MR list). If the Maintainer has any comments, then they work with the Author to clarify. It is the responsibility of the Maintainer to find another Maintainer to look at a merge request that is assigned to them if they are unable.

NOTE: If you are working on a merge request that requires a response quicker than the SLO, please @ mention the gitlab-org/distribution group in order to alert the Distribution team. The team will exercise best effort in handling these requests.

Reviewers

By default, every Distribution Engineer team member who is not a Maintainer on a project should consider themselves a Reviewer. You are encouraged to spend some of your time looking at Merge Requests in any of our projects that have the "workflow::ready for review" label.

Reviewers are responsible for working with contributors to ensure that changes meet our standards, before approving and passing them on to a Maintainer for final review and merge. Reviewers should confirm that the Merge Request addresses the problem the linked Issue describes. Reviewers are also responsible for verifying that all applicable Merge Request checklist items have been completed. In situations where a checklist item is not applicable, Reviewers should verify that the item is indeed not necessary. When you encounter a situation as Reviewer where you are unsure whether something meets our standards, ping a Maintainer directly within the Merge Request with the question.

Additionally, in the spirit of "everyone can contribute", anyone who is interested is encouraged to be a Reviewer. There should be no barrier preventing anyone from reviewing available merge requests. We encourage any interested party to participate.

Anyone who plans on actively participating in the Reviewer process is encouraged to update their entry on the team page.

When looking for merge requests to review, consider checking the Merge Request Report to see what reviews are in danger of breaching the SLO.

Maintainers

Project Maintainers are encouraged to ensure that Reviewers, and in particular Reviewers who have designated themselves enrolled in the Reviewer mentorship program, look at a Merge Request before they spend time on it. There are times when it makes sense for a Maintainer to not wait for a reviewer, so judgment should be used here. For example, we do need to keep the SLO in mind. If an MR is in danger of missing that deadline, a Maintainer should not hesitate to respond.

Assigning Merge Requests

To help achieve all of this, we should urge contributors to not assign merge requests to an individual when looking for initial review, unless there is a specific reason someone should look at a merge request. Rather, the merge request should have the "workflow::ready for review" label applied, and a Reviewer will add themselves under the Reviewers section when they are beginning to look into it.

If a merge request is assigned directly to you as a Maintainer without prior review, you are encouraged to assign it to an available Reviewer. If a merge request is assigned directly to you as a Reviewer, use your judgment. If you will not be able to work on it soon, try and find another Reviewer to take a look.

Service-level objective

When looking for a merge request to work on, consider the GitLab Review-response SLO. Anything in danger of breaching that deadline should be looked at first.

Due to the load on the Distribution team, the SLO is longer than that of the rest of the company:

- From GitLab team members: Review-response SLO < 4 business days

- From authors of Leading Organizations: Review-response SLO < 6 business days

The team aims to return to the company-wide SLO standard dependent upon expanding the team and paying down technical debt.

Review iteration

Once the merge request is in review, the "workflow::in review" label should remain on the merge request as the Reviewer/Maintainer and Author iterate through feedback.

When the merge request is ready to be handed back for changes or further review, ensure that the individual responsible for the next step is assigned and signal the handoff with a fresh comment mentioning the individual in the Merge Request.

NOTE: By default, Authors should handoff to the Reviewer/Maintainer who previously reviewed the merge request. If that individual is listed as away in their status for longer than two days, then please @ mention the gitlab-org/distribution group in order to alert the Distribution team for a new reviewer.

Squash and merge

Distribution-owned projects enable squash and merge by default. This feature combines all of the merge request's commits into one commit before merging, ensuring a clean history on the target branch.

Because the squash and merge feature is set to encourage, authors can still disable the option if desired. For example, merge requests with only one commit would not necessarily benefit from a squash prior to merge.

References

1. Distribution team projects - The full list of projects the Distribution team maintains
1. Engineering Projects - The full list of supported GitLab projects. Clicking on the project name will bring you to the list of Maintainers and Reviewers for each project.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/training.md

Common links

Distribution Team Handbook

Purpose

The purpose of team training is to introduce the work done to the rest of the team. It also allows the rest of the team to easily transition into new features and projects and prepare them for maintenance. Training is public and available for everyone wanting to get familiar with the topics.

Training should be:

- Providing a high level overview of the invested work
- Describing the main challenges encountered during development
- Explaining the possible pit-falls and specificities

Training should not be:

- Replacement for documentation
- Replacement for writing down progress in issues
- Replacement for follow up issues

Simply put, the training is a summation of: notes taken in issues during development, programming challenges, high level overview of written documentation. Your team member should be able to take over the maintenance or build on top of your feature with less effort after they have been part of the training.

Note Do not shy away from being technical in your training. You can ask yourself: What would have been useful for me when I started working on this task? What would have helped me be more efficient?

Efficiency of the training

In order to see whether the training is efficient, Distribution lead will rotate team members on projects where training was done. For example, if the feature requires regular releases, the person who gave the training will be considered a tutor. Different team member will follow the training and documentation and will ask the original maintainer for help. The new person responsible is now also responsible for improving the feature. They are now also responsible of training other team members.

Training Listings

- DIS001 GitLab Pivotal Cloud Foundry Tile
- DIS002 GitLab Terraform Module
- DIS003 Kubernetes
- DIS004 Omnibus
- DIS005 PG HA
- DIS006 GitLab QA project
- DIS007 Let's Encrypt with the omnibus-gitlab package
- DIS008 GitLab Helm Chart Operator
- Contributor introduction to Omnibus-GitLab

FAQ

Q: Isn't this double work?

A: No. The training should be prepared while documenting the task.

Q: Won't this slow me down?

A: At the beginning, possibly. However, every hour of the training given will

multiple the value of it by the amount of team members.

Q: Isn't it more useful to let the team check out the docs and ask questions?

A: In an ideal world, possibly. However, everyone has a lot of tasks assigned and they might not be able to go through the docs until they need to do something. This might be months later and you, as a person who would give the training, might not be able help efficiently anymore.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/triage.md

Common links

Engineering Team Triage

Triaging Issues

Terminologies

Triaging: Triaging issues involves investigating and applying labels and milestones to issues. Partially Triaged issues: Issues are considered partially triaged if they have been assigned for scheduling or awaiting feedback labels.

Fully Triaged issues: Issues are fully triaged when they have been assigned a Milestone, even if it is Backlog. They should have appropriate group, priority and product category labels applied.

Note: An issue that has been assigned to a user, but has no milestone, is not triaged, but is considered the responsibility of that user, and is not part of our triage queue at this time.

Issue severity

See the CE documentation for an explanation of the severity labels.

Label glossary

| Label | What it means | How to handle it |

| - | - | - |

| awaiting feedback | We've asked more information from the user | If no reply has been received in two weeks, the issue can be closed. |

| maintainer-discussion | Issues for further discussion by project Maintainers | Projects maintainers should review status and provide input within 2 weeks. |

| needs investigation | Information has been provided by the user, but is waiting on the team to further dive in | The team member who added the label should try to find some time to investigate or engage other team members within 4 weeks. |

During triage, an appropriate group:: label should be applied to the issue.

group::distribution::build for Distribution Build team responsibilities

group::distribution::deploy for Distribution Deploy team responsibilities

group::distribution for Distribution team shared responsibilities

The otherwise appropriate group to assign based on GitLab Features by Group

During triage additional labels should be added to indicate what part of the product is impacted by the issue. Descriptions for the labels that Distribution often uses can be found the Distribution Frequently Used Labels Page.

Resources

Issues for triaging can be identified using the following criteria:

They have no milestone

They have no assignee

They do not have any of the following labels applied:

awaiting feedback

for scheduling

maintainer-discussion

needs investigation

Seeking community contributions

Such issues can be listed using the issues filter

Process

Distribution team implements issue triaging on a rotating schedule, where each person in the team gets assigned to do it for one week. This is done in the order of joining the company. The process which one follows while on issue triage duty can be summarized as follows

1. The previous team member on triage duty should have created and assigned the weekly meta issue to you at the end of their week. The issue title should be Issue triage rotation week of <starting date>
1. We follow the policy of closing an issue if it has been 14 days since awaiting feedback label was added and no response was received from submitter. Check out the issue list with awaiting feedback label for such issues and close them with the "for issues with no reply" response.
1. Check out the issues to be triaged and assign appropriate labels to them. While it is normal for some issues to demand a bit of research to get to the bottom, do keep in mind issue triaging need not end up in issue resolution. Triageing is intended only to identify and classify issues so that appropriate action can be taken on them, based on priority.
1. If an issue doesn't fall directly under the domain of the project it is logged in (for example, needs changes to gitlab-rails application, but was made in the omnibus-gitlab project), move the issue to the appropriate issue tracker and add the team label which may be the most appropriate. If you can't identify which project's tracker should the issue reside or which team's label should be applied, you can ask for Quality team's help for doing it by mentioning @gitlab-org/issue-triage in the comments.
1. If an issue doesn't deal with the code base or work flow of the project

but is more of a request for help for installing/configuring/troubleshooting a GitLab instance, close the issue using the "problems not related to package installation and configuration" response.

1. If an issue doesn't have all necessary information to successfully triage the issue, request the information using the "issues that lack enough information" response and add the awaiting feedback label.
1. If an issue couldn't be triaged in reasonable time, add the needs investigation label to it.
1. Issues occasionally would be best inspected by a project maintainer. When this occurs, label them with maintainer-discussion. Example cases where maintainers should be notified with this label:
 1. An issue was accurate at the time of creation, but is questionable at this time.
 1. An issue was not accurate or in line with our vision at the time, but may be now.
 1. The viability of the issue is of interest, but may not be reasonable as proposed.
1. If an issue is identified as a valid one, make it partially triaged by assigning For Scheduling label so that it gets scheduled for one of the upcoming milestones (or even Backlog milestone). Also apply the severity labels as applicable.
1. Try to engage the community using Seeking community contributions label where appropriate.

The issues that are good

candidates for community contribution should have low severity and priority or have a valid workaround. They must be

in workflow::ready for development state. If the required change is simple and trivial use quick win label.

See Omnibus examples.

1. If an issue is a valid issue, and there is a valid workaround. Close the issue with details on the

workaround, and open a follow up issue to fix the bug. Ensure the issue includes:

1. Details on how to recreate the bug
 1. How to workaround the issue
 1. What the expected behavior is
 1. The user issue as a related issue
 1. The appropriate priority and severity
1. Once an issue is triaged, either partially or fully, add the link to that issue as a comment to the meta issue. Preferably in the format of <action> <url> where <action> can be moved, closed, triaged, resolved or marked as needs investigation. This helps in keeping track of the issue triaged per week. Related issues feature of GitLab should not be used for this purpose as it generates an incorrect association between the issues.
 1. Use the /spend quick action feature of GitLab to track the time you spent in triaging issues. There are no hard limits for this, but a reasonable amount of time would be 3 to 5 hours a week.
 1. Since issues piling up is almost inevitable in any software project, we should try to bring down the issue count as much as possible. This mainly involves tackling existing backlog of untriaged issues.
 1. During the next weekly meeting, inform the team about the triage week - link to the meta issue, number of issues triaged and time spent. And discuss if you think something should be changed regarding the process. Also, create a similar header for the next presenter as an agenda item for the next week's meeting in the meeting doc.

Response templates

Copy and paste into issues where appropriate

For problems not related to the GitLab codebases

If someone is asking for support in our projects, point them to the correct place to look

text

We are sorry you are having troubles. The provided issue description seems to indicate that the problem is not related to this project. Commonly this indicates other troubles such as network connectivity or filesystem permissions.

For this reason, I will close this issue and recommend checking out how to get further help on the GitLab website.

/close

For problems related to GitLab code but not specifically Omnibus

If someone is asking for help with a bug that seems related to GitLab code other than Omnibus

text

We are sorry you are having troubles. The provided issue description seems to indicate that the problem is not related to Omnibus.

For this reason, we are moving this report to a more appropriate issue queue. Please review the bug templates for the new project in case they require additional information to help diagnose the problem.

We also recommend checking out how to get further help on the GitLab website.

For issues that lack enough information

If someone opened a ticket without enough information, make sure they use the Bug template, and fill it in

text

We can't reproduce the issue with the information you provided here.

Can you please use our Bug template to help gather more details?

1. Click on the pencil icon near the issue title to edit the description
1. From the Choose a template drop down, select the Bug template
1. Read the template, and provide as much information as you can that we ask for
1. Click on the Save changes button to apply your changes.
1. Add a comment mentioning you updated the description.

/label ~"awaiting feedback"

For issues with no reply

If an issue has been labeled awaiting feedback for two weeks, and we haven't received a response, it can be closed

text

We haven't heard back from you, so we're going to go ahead and close the issue.

If you're still experiencing the problem, please re-open the issue and provide the requested information.

/close

For issues closed with no reply with new comments from a different reporter

If an issue was closed for no reply and someone comments who is not the original reporter, we ask them to open a new issue. Be sure to tag the contributor who made the comment.

text

Thank you for letting us know about your issue COMMENTOR. Unfortunately, this issue was already closed. Please open a new issue following the BUG template and mark this closed issue as related.

For issues where a maintainer should review the current viability

If an issue appears to need review directly by a project maintainer to ascertain relevance,

text

I'm going to ask that this issue be reviewed by the project maintainers directly.

This is so that we can make the most accurate decision regarding further work and viability.

/label ~maintainer-discussion

Onboarding to Triage

We start looking to onboard new Distribution team members to Triage duty after 3 months. This can be sooner if a team member feels they are familiar enough with the team processes to start working on triage.

To onboard to triage, a team member will shadow an experienced triage team member for a week. Expectations are:

The mentor and shadow were made aware and agreed to mentoring prior to the triage week.

The shadow reads the triage process document in advance.

The mentor and shadow have a synchronous call earlier in the triage week for the mentor to go

over how they approach triage.

The shadow attempts to triage a few issues asynchronously during the week.

A followup sync call is scheduled for later in the week for mentor and shadow to try triaging some issues together.

A shadow may need multiple weeks on shadow before they feel comfortable on a solo triage week.

Triaging Pipelines

Pipeline failures are a shared team responsibility and need to be handled as soon as possible by whomever is available. That said, the team member on triage duty has a responsibility to follow up on any failures which occurred and provide a summarized tally in the Triage notes.

Note that we are in the process of automating issue creation for every time a critical pipeline fails. Some pipelines failures are notified in our gdistribution Slack channel, but do not yet have issues automatically created for it. See this epic to track ongoing work.

Recommended workflow for pipeline failure triage

1. Optionally, share on the Slack message any initial thoughts, or leave a comment indicating your action plan.

1. Check if an issue was automatically created for it. If not, please create one.

1. Link the issue in the initial Slack message so that everybody contributing to solve the problem can use the issue as a single source of truth.

1. Once you started investigating the issue, add the pipeline failure::under investigation label to it.

1. Comment on the issue with the following format: @gitlab-bot retryjob <job-id>. This will trigger triage-ops bot to retry the given job without requiring you to have Maintainer permissions on the project.

1. Repeat the retry command as needed. If this causes the failed job(s) to pass, close the issue. If not, either fix the root cause manually and retry again, or create a new related issue to track the root cause.

1. If you managed to mitigate the problem and make the pipeline pass, but the issue could comeback in the future, then:

Write a comment explaining which actions did you take to mitigate it.

Check if there's an existing follow-up issue to investigate/implement a definitive fix for the problem, and link it to this pipeline issue failure. If such a follow-up issue does not yet exist, create one and link it.

If you had to restart a release which had already been stopped, after you fix the pipeline, manually stop the environment with @gitlab-bot retryjob <job-id of stop-review>. This is necessary because that job won't be automatically re-triggered, and dangling releases can cause ELBs quota exhaustion. See chart issue 5326.

1. Failures requiring follow up issue(s) to fix pipelines that are still in a broken state should also be noted to increase team awareness. Those issues should be labeled with Broken Pipeline.

1. Failures caused by upstream failure should be labeled with Upstream bug and linked with the related upstream issue.

1. Before closing the issue, don't forget to set the milestone and double check that you have added pipeline failure::under investigation. This is important for metric purposes.

Details on the various pipelines and jobs implemented by different projects under Distribution are listed below:

1. omnibus-gitlab

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/workflow.md

Common links

Engineering Team Workflow

Summary

Distribution team members are expected to:

- Be kind in their interaction with the rest of the community and other teams
- Ensure that fixing red main branch in projects takes the highest priority
- Pick items to work on from the project scheduled queues
- Define test plans for changes not covered by integration tests
- Label issues and merge requests to track engineering metrics

Groups

The distribution team is comprised of two groups, Distribution:Build and Distribution:Deploy.

Distribution Build tasks

- Maintain all team pipelines for all Distribution projects
- Research for the support of new clouds, platforms, architecture, and components
- Access controls, permissions, and CVE patches
- Team infrastructure/resource management
- Dependency updates
- License management
- Submissions to Partners for validations/certifications

Distribution Deploy tasks

- Initial installation and composability
- Upgrades / Downgrades
- Scaling deployments
- Migration between platforms or providers
- Data lifecycle management
- Secure configuration & communication
- Research of clouds and platforms for integration to existing tools

Work prioritization

Work to be completed by the Distribution team members who are not currently

acting as the Distribution DRI is prioritized as follows:

Priority level	Work item
-	-
1	Unblock in-review ~priority::1 Merge Requests
1	Pick up ~priority::1 Merge Requests for review
1	Work on in-progress ~priority::1 Deliverable Issues
1	Pick up available ~priority::1 Deliverable Issues
2	Unblock remaining in-review Merge Requests
3	Work on in-progress ~priority::2 Deliverable Issues
3	Pick up SLO-breaching Merge Requests for review
4	Pick up SLO-near-breaching Merge Requests for review
5	Pick up available ~priority::2 Deliverable Issues
6	Work on in-progress ~priority::3 Deliverable Issues
6	Pick up available ~priority::3 Deliverable Issues
6	Pick up SLO-non-breaching Merge Requests for review
7	Work on in-progress ~priority::4 Deliverable Issues
7	Pick up available ~priority::4 Deliverable Issues

Use this prioritization outline as a general guide when determining what to do each day. This list helps direct work toward overall team priorities and goals laid out by the team managers.

Workflow Summary

Distribution groups use the GitLab product development flow and labels in principle, we usually skip below phases due to the nature of our work:

Validation phase 3: Design

Validation phase 4: Solution Validation

Planning process

All the planning dashboards are bookmarked on the gdistribution Issue dashboards folder.

Quarterly planning

On the 3rd month of the quarter:

1. PM & EM & Staff Engineer: By 1st week, they draft OKRs for the upcoming quarter.
1. OKR DRI: From 2nd week, they pick up a OKR and start scoping with other team members or stakeholders.
1. All Engineers: From 2nd week, they start contributing to any OKR scoping they are interested in.
1. EM: By the end of 2nd week, they decide whether any reserved planning days require per the current status of scoping.
1. Everyone: At the end of the month, the OKRs and their scoping should be finalized.

Milestone planning

At the end of the current milestone, every engineer is expected to update their assigned issues and roll over uncompleted ones to the next milestone for review.

Engineering manager and Product manager are responsible for scheduling work that is a direct responsibility of the Distribution team.

Any unassigned issue is available for anyone to work on. You can show your interest in working on a specific task by leaving a comment in the issue.

To do so, comment must contain the date on which you will commence the work. If the comment does not contain the date, or the date has passed, the item is free to be picked up by anyone again.

Add Unplanned work to current milestone

Sometimes, there are works that were not originally planned but require urgent attention, anyone could pick up and work on the reasonable ones. A typical workflow is:

1. Assesses the work's urgency with your best judgement, the urgency usually could be determined by active incident, internal and external customer impact, SLA and policies, and deadline.

1. If it is a reasonable request, assigns Unplanned and all other required labels, and works on it per the work prioritization.

1. If it is not, explains the reason to the requestor and provides possible alternatives, escalates to EM&PM when necessary.

1. If not sure, asks for clarification from the requestor, and seeks advice from EM&PM when necessary.

Backlog reviewing

To be updated

Required labels

Besides the GitLab product development flow labels, there are a number of additional required labels applied at any time to Epics, issues and merge requests (items):

`group::distribution` - Items specific to, or authored by Distribution team. It is a scoped label to be applied to all Distribution subgroups items until further guidance.

`group::distribution::` - Items specific to, or authored by one of Distribution subgroups. They are nested scope labels, and mutually exclusive, but can be used with `group::distribution` scoped label together.

`group::distribution::build` - Items specific to, or authored by build group.

`group::distribution::deploy` - Items specific to, or authored by deployment group.

`FY(Year in two digitals):: (1-4)` - scoped label to indicate efforts targeted for release within a quarter. i.e., `FY24::Q2`.

There are also a number of additional required labels under certain scenarios:

Unplanned - Urgent requests that were not originally planned for the current milestone. Work on these items is

usually event driven - another team requires help, regression affecting users, or technical debt that is causing inefficiency in the team.

spike - Issues which primarily involve research to understand options and the breakdown of future deliverables. Spikes are often the first issue in a new Epic where the output defines additional issues and order of serial/parallel work.

In addition to the labels outlined above, see also workflow labels used during merge request review and labels used during issue triage.

Priority definition

To be updated

Distribution DRI

In order to minimize disruption and context switching for team members, Distribution designates one engineer on a weekly rotation basis (DRI, Directly Responsible Individual), who will be responsible for the following duties during their normal office hours. For urgent requests outside of those hours, it will be handled via the on call process.

Expectation

Similar to Development Escalation Process, the DRI is not solely responsible for a resolution of any requests, they should engage any SME and escalate to engineering manager and/or product manager whenever requires. In addition to that, there is also no expectation that the DRI can complete the same amount of Deliverable work during the week.

Duties

The Distribution DRI works on the following areas per the order of the list.

During the week

1. Support incidents escalated from production.
1. Support customer requests
 - Support help requests for Build and Self-managed
1. Answer or redirect questions in gdistribution Slack channel.
1. Perform issue triage.
1. Respond to @gitlab-org/distribution group mentions in GitLab.
1. Optional: Work on deliverables for the current milestone or other Distribution related tasks.

Handover

1. Close the assigned DRI issue.
1. Rotate your name to the end of the shift assignment list.
1. Create a new issue for the next shift and assign it to the next team member on the list. The issue title should be Distribution DRI rotation week of <starting date>. Use the Distribution-

DRI template to fill in the description.

1. For requests still active at the time of handoff, the DRI should use best judgement and consider one or several options:

Close the request with a documented mitigation and open a follow-up issue.

Add a comment that hands off the request to the next week's DRI.

Continue to act as SME, though handoff to the next DRI with clear documentation to share knowledge is strongly preferred.

Pass information to the next DRI in a synchronous conversation.

Other team members

When you are not on DRI duty, please consider the following when the request is not Stuff that should Just Work:

1. Redirect any active incident and deployment blocker request to the Distribution DRI during their normal office hours, or take the request if you are the first available team member

1. Redirect any Slack DM to the channel `gdistribution`

1. Redirect any GitLab direct mention to `@gitlab-org/distribution` group

1. Redirect any other requests to How to work with Distribution

Iteration

Use iteration to better control scope and deliver measurable value in each release. A timebox measurement process would ensure that if expected progress isn't achieved, there's a procedure to follow.

If a project goes more than 2 milestones without delivering on the success criteria within the defined MVC/issue, a detailed evaluation/retrospective should be performed.

If the project goes 3 milestones without meeting success criteria, then a larger evaluation should be performed with the section leaders.

The goal is to catch scope creep, ensure we are iterating in ways that deliver measurable value, and establish circuit breakers to address earlier as needed.

Distribution dependency maintenance policy

Distribution team follows below dependency maintenance policy in order to achieve our technology vision.

Distribution aims to add support to newer release of Distribution managed dependencies within 3 to 5 milestones after their original release unless specified below otherwise.

Kubernetes release support policy

Distribution team follows kubernetes release support policy for supporting Kubernetes releases.

Distribution aims to add support for newer releases of Kubernetes within 3 months of their release.

OS

All the operating systems supported by GitLab and their EOL policy are listed in the Supported OS doc page and installation page.

For new OS release, Distribution team aims to provide Linux package support per the below timeline:

OS	Expected to be supported by
Ubuntu LTS release	within 2 milestones after OS release date
RHEL(& RHEL Clones) major and minor release	within 3 milestones after OS release date
Debian LTS and minor release	within 3 milestones after OS release date
All others minor release	within 3 milestones after OS release date
All others major release	within 4 milestones after OS release date

Interviews

See the hiring-process project for information on conducting interviews for positions on the Distribution team.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/maintenance/_index.md

Common links

Distribution Team Handbook

Infrastructure

As part of the team tasks, team has responsibility towards the following nodes/tasks:

dev.gitlab.org: This internal GitLab instance runs nightly CE packages and is used for building official packages as well as hosting security release related MRs before publishing. Details of the node as well as the maintenance tasks can be found in the dev.gitlab.org specific docs.

Build Machines: Runner manager machines that spins up machines that are used by various CI jobs for building and publishing packages. Details of the node as well as the maintenance tasks can be found in the build machines specific docs

packages.gitlab.com: This is a self-hosted package server managed by GitLab Infrastructure team. It's used by Distribution team to ship GitLab CE and EE omnibus-gitlab packages along with Verify team to ship gitlab-runner packages to users. GitLab CE and EE packages are built via our CI pipeline on dev.gitlab.org.

Distribution uses the package server as a tool and doesn't have any maintenance tasks associated with it. The package server is currently deployed on our own infrastructure from a package provided by Packagecloud.io. If the Infrastructure team requires help, Distribution team should provide a best effort to resolve issues.

Keeping SSH public keys gitlab.com and dev.gitlab.org up-to-date: The CI configuration of omnibus-gitlab uses public SSH keys of these servers during execution. The keys are stored in the codebase (in support/knownhosts) and it is the responsibility of the team to update them in case any of them changes. To do so:

```
console
bundle exec rake infrastructure:knownhosts
git add support/knownhosts
git commit -m "Update SSH keys"
```

Please note that you need to push this change as a separate MR and ask the maintainer for review.

External Services

As part of the team tasks, the team uses the following external services:

dependencies.io: This is used to automatically update software components required by GitLab. See the specific docs for how to work with this service.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/maintenance/arm.md

Hardware Provider

AWS Graviton is the current provider for ARM64 and Raspberry Pi runner instances. All distribution team members should be able to log in with their own credentials.

Teams Working on ARM Related Tasks

Several GitLab teams collaborate to provide support for ARM:

1. Distribution manages the ARM runner provider and issues with build pipelines.
1. Developer Relations team and support teams answer questions and provide community assistance in the forums.
1. Verify CI would help build packages for ARM runners.

Failure Notifications

Build failures for master, stable branches, and tags are sent to the distribution team slack channel. Developers receive failure notices via e-mail for pipelines triggered from their feature branches as they would normally for any other branch.

Frequently Asked Questions

What GitLab packages get built for Raspberry Pi?

Due to memory requirements we do not currently recommend the Raspberry Pi as a production platform. Due to this, we only build packages for the Community Edition.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/maintenance/build-infrastructure.md

Common links

- Distribution Team Handbook
- Distribution Team Infrastructure and Maintenance

Build Infrastructure

Distribution uses GitLab CI to build GNU/Linux packages, Docker images, AWS AMIs, and other artifacts that can be used to install GitLab in user's machines. Many of these build processes require greater computing power and resources, and hence Distribution maintains their own infrastructure to provide these resources.

Runner manager machines

The Distribution team maintains 2 runner manager machines - one to be used in GitLab.com and the other to be used in dev.gitlab.org. The configurations of runner managers are maintained in the chef-repo project, and the secrets to be used together with these configurations are maintained in Vault.

The runners are named using the syntax runners-manager-distribution-(com|dev)-(blue|green)-<count>.

NOTE: There is no blue/green deployments configured for these runner managers yet, but issue 1138 discusses using a blue/green deployment for easier upgrades of GitLab Runner in these runner manager machines. Currently, only a single "blue" runner manager is available for each of gitlab.com and dev.gitlab.org.

Build machines

Both com and dev runners spin up identical machines using the docker-machine executor with autoscale configuration. The different type of machines spun up by them are as follows:

1. distribution-(com|dev)-amd64-runner-gcp - These runners spin up

n1-highcpu-32 machines with 100GB SSD in GCP that are used for building omnibus-gitlab GNU/Linux packages for amd64 architecture.

1. distribution-(com|dev)-arm64-runner-gcp - These runners spin up t2a-standard-8 machines with 100GB SSD in GCP that are used for building omnibus-gitlab GNU/Linux packages for arm64 architecture.

1. distribution-(com|dev)-arm64-runner-aws - These runners spin up m6g.2xlarge machines with 100 GB SSD in AWS that are used for building omnibus-gitlab GNU/Linux packages for armv7 architecture (Raspberry Pi). They are used because GCP's T2A machines don't support 32-bit instruction set that is used to build armv7 packages while AWS's Graviton does.

Maintenance tasks

Requirements:

- Access to the node.
 - SSH access is granted by adding public SSH key and build group access to the Chef Repo. See [this example account](#).
- Access to specific folders in Chef Vault.
- Access to merge into master on th